Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Специальность «Программная инженерия»

Кафедра инженерной психологии и эргономики

Учебная дисциплина «Компьютерные системы и сети»

ОТЧЕТ

по лабораторной работе №5
«Файловое хранилище»

Подготовила: Плехова Е.В.

Проверила: Болтак С.В.

Минск 2026

Задание: Необходимо реализовать удаленное файловое хранилище, предоставляющее REST API по протоколу HTTP. При реализации разрешается использовать веб-фреймворк, реализующий HTTP-роутинг и работу с HTTP-вызовами.

Путь URL запроса определяет логическое расположение файла в хранилище: http://storage.com/path/to/file.txt – описывает файл file.txt, находящийся по пути /path/to.

Список поддерживаемых функций:

– загрузка файла в хранилище с перезаписью с помощью метода PUT;

– получение файла из хранилища с помощью метода GET;

– получение списка файлов каталога с помощью метода GET (в качестве формата данных для ответа рекомендуется использовать JSON);

– получение информации о файле (размер в байтах и дата последнего изменения) в виде HTTP-заголовков без получения содержимого файла с помощью метода HEAD;

– удаление файла/каталога из хранилища с помощью метода DELETE.

Необходимо корректно использовать коды состояния HTTP.

Для тестирования созданной службы использовать утилиту curl или аналоги (Postman, HTTPie, etc). Получение файла или содержимого каталога также должно быть возможным через браузер.

Дополнительное задание:

Реализовать функцию копирования файлов в новое расположение.

Решение.

В ходе выполнения лабораторной работы было реализовано файловое хранилище на языке C# с использованием веб-фреймворка ASP.NET Core Minimal API на платформе .NET 8.0 (дополнительное задание также реализовано). Для тестирования используется утилита curl.

Физически файловое хранилище находится на моем ПК – это папка Storage (рис 1).

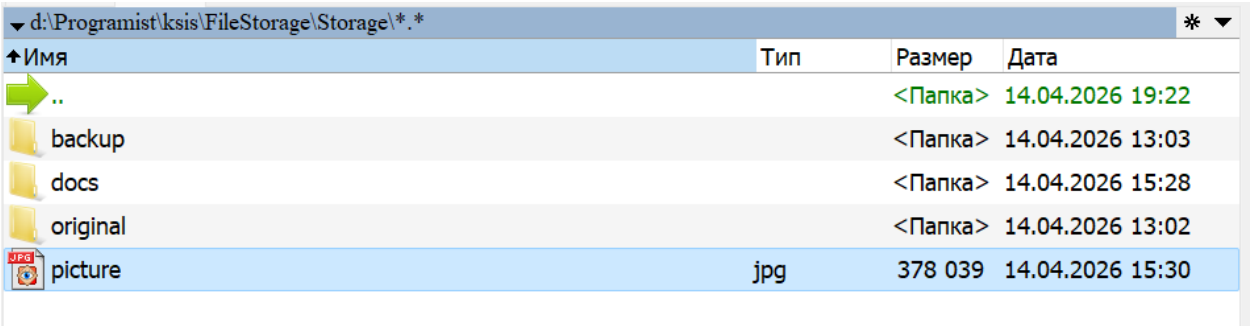


Рисунок 1 – Файловое хранилище

Логически хранилище доступно по адресу http://localhost:5166, где путь в URL соответствует пути к файлу или папке внутри хранилища. Отображение хранилища в браузере представлено на рисунке 2.

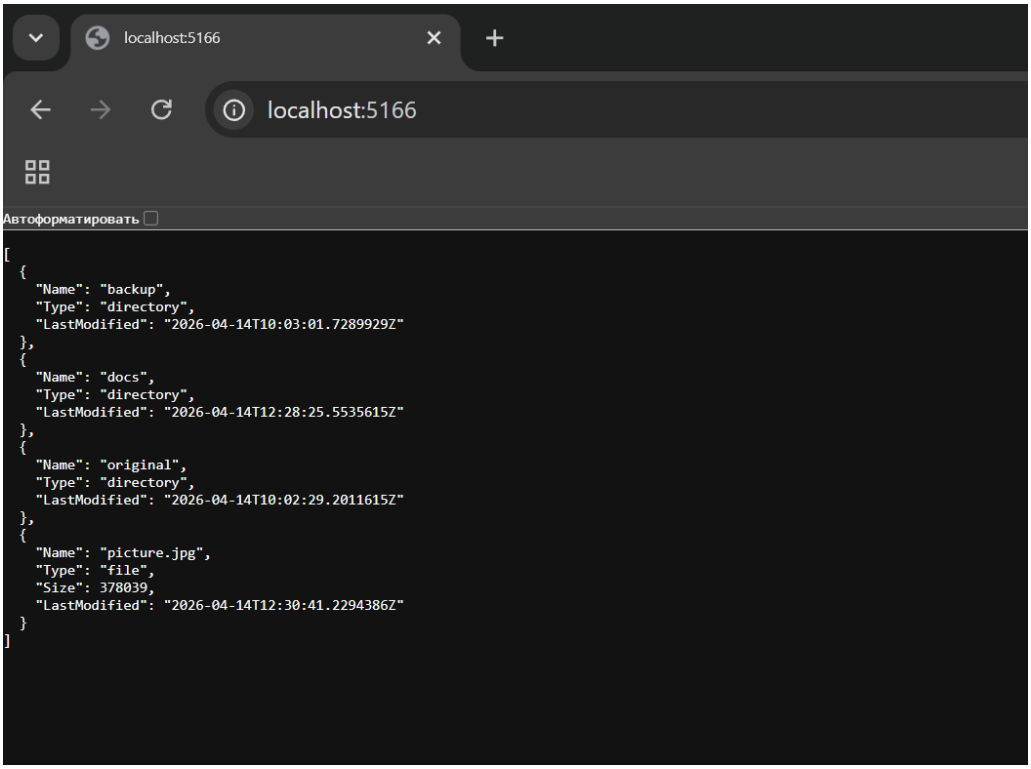
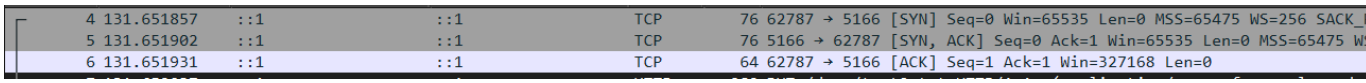


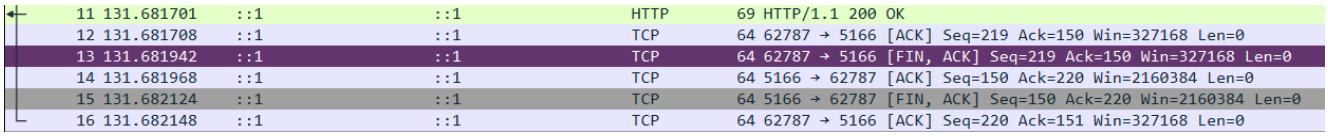
Рисунок 2 – Файловое хранилище в браузере

При запуске приложения веб-сервер Kestrel начинает прослушивать порт 5166, переходя в состояние ожидания входящих соединений. Установка TCP-соединения происходит при поступлении HTTP-запроса от клиента curl (тройное рукопожатие представлено на рисунке 3). После обработки запроса и отправки ответа TCP-соединение закрывается (рис 4).



4	131.651857	::1	::1	TCP	76 62787 → 5166 [SYN] Seq=0 Win=65535 Len=0 MSS=65475 WS=256 SACK_
5	131.651902	::1	::1	TCP	76 5166 → 62787 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65475 W
6	131.651931	::1	::1	TCP	64 62787 → 5166 [ACK] Seq=1 Ack=1 Win=327168 Len=0

Рисунок 3 – Установка соединения



11	131.681701	::1	::1	HTTP	69 HTTP/1.1 200 OK
12	131.681708	::1	::1	TCP	64 62787 → 5166 [ACK] Seq=219 Ack=150 Win=327168 Len=0
13	131.681942	::1	::1	TCP	64 62787 → 5166 [FIN, ACK] Seq=219 Ack=150 Win=327168 Len=0
14	131.681968	::1	::1	TCP	64 5166 → 62787 [ACK] Seq=150 Ack=220 Win=2160384 Len=0
15	131.682124	::1	::1	TCP	64 5166 → 62787 [FIN, ACK] Seq=150 Ack=220 Win=2160384 Len=0
16	131.682148	::1	::1	TCP	64 62787 → 5166 [ACK] Seq=220 Ack=151 Win=327168 Len=0

Рисунок 4 – Закрытие соединения

Протестируем функции файлового хранилища.

1 Загрузка файла в хранилище с перезаписью с помощью метода PUT:

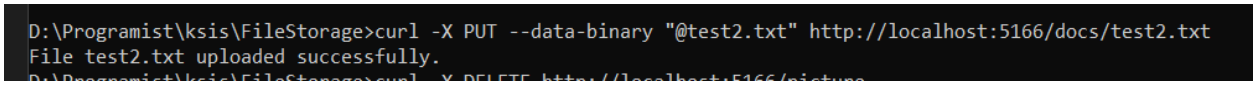


Рисунок 5 – Загрузка файла в хранилище

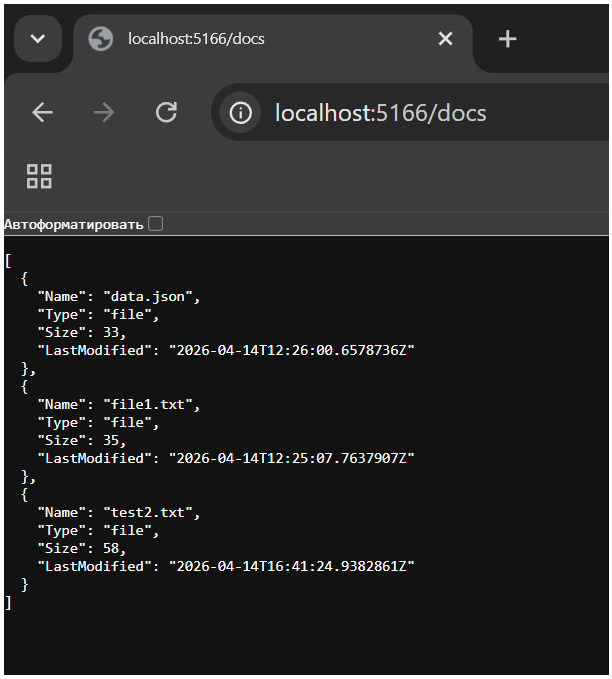


Рисунок 6 – Содержание папки «docs» в хранилище

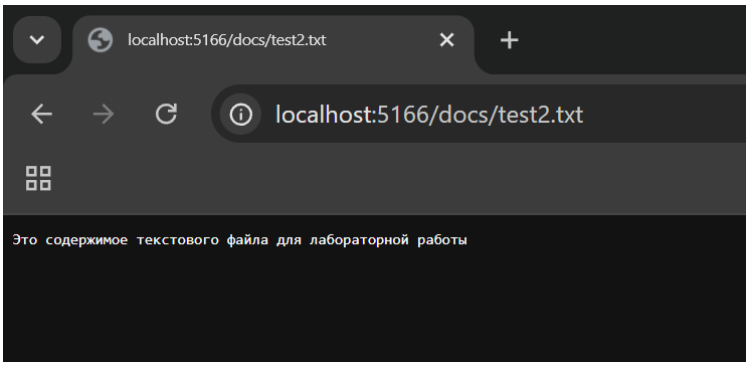


Рисунок 7 – Содержимое загруженного файла

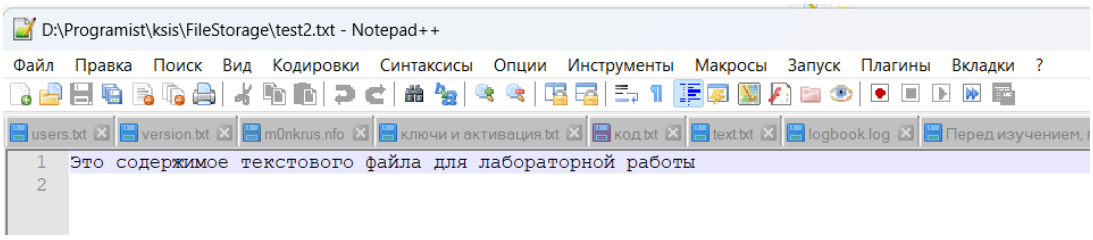


Рисунок 8 – Файл «test2.txt» в хранилище

Далее изменим содержимое файла и перезапишем его с помощью метода PUT.

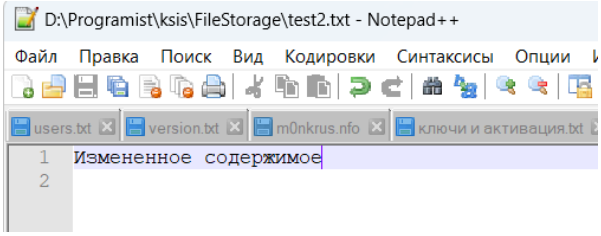


Рисунок 9 – Новое содержимое файла

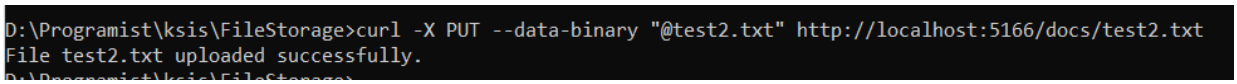


Рисунок 10 – Перезапись файла

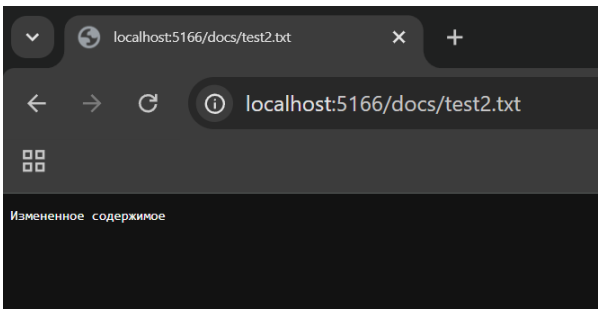


Рисунок 11 – Новое содержимое файла в хранилище

2 Получение файла из хранилища с помощью метода GET

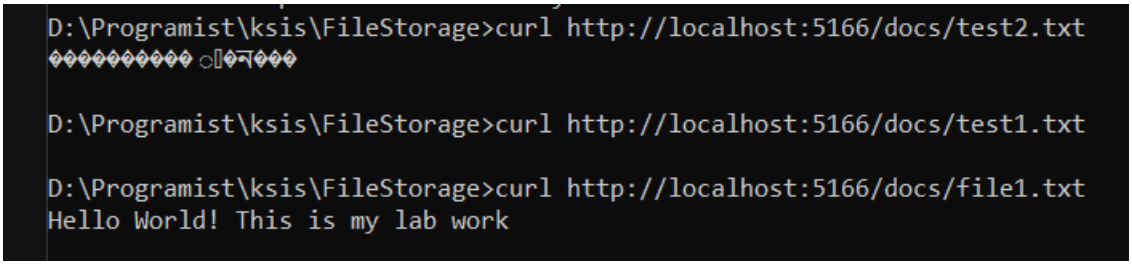


Рисунок 12 – Получение файла из хранилища

Пояснение: файл «test2.txt» содержит русский текст, поэтому отображается некорректно. Файл «test1.txt» не существует, поэтому мы получили пустую строку. Файл «file1.txt» существует и содержит текст на английском языке, который мы и получили. Утилита curl использует метод GET по умолчанию, поэтому его можно явно не указывать.

3 Получение списка файлов каталога с помощью метода GET

```
D:\Programist\ksis\FileStorage>curl http://localhost:5166/docs
[
  {
    "Name": "data.json",
    "Type": "file",
    "Size": 33,
    "LastModified": "2026-04-14T12:26:00.6578736Z"
  },
  {
    "Name": "file1.txt",
    "Type": "file",
    "Size": 35,
    "LastModified": "2026-04-14T12:25:07.7637907Z"
  },
  {
    "Name": "test2.txt",
    "Type": "file",
    "Size": 23,
    "LastModified": "2026-04-14T17:07:47.2231887Z"
  }
]
```

Рисунок 13 – Получение списка файлов каталога

4 Получение информации о файле в виде HTTP-заголовков без получения содержимого файла с помощью метода HEAD

```
D:\Programist\ksis\FileStorage>curl -I http://localhost:5166/docs/test2.txt
HTTP/1.1 200 OK
Content-Length: 23
Content-Type: text/plain
Date: Tue, 14 Apr 2026 17:17:00 GMT
Server: Kestrel
Last-Modified: Tue, 14 Apr 2026 17:07:47 GMT
```

Рисунок 14 – Получение информации о файле

5 Удаление файла/каталога из хранилища с помощью метода DELETE

```
D:\Programist\ksis\FileStorage>curl -X DELETE http://localhost:5166/docs/test2.txt

D:\Programist\ksis\FileStorage>curl -I http://localhost:5166/docs/test2.txt
HTTP/1.1 404 Not Found
Date: Tue, 14 Apr 2026 17:19:22 GMT
Server: Kestrel
```

Рисунок 15 – Удаление файла

```
D:\Programist\ksis\FileStorage>curl -X PUT --data-binary "@haski.jpg" http://localhost:5166/images/picture.jpg
File picture.jpg uploaded successfully.
D:\Programist\ksis\FileStorage>curl  http://localhost:5166/images/
[
  {
    "Name": "picture.jpg",
    "Type": "file",
    "Size": 378039,
    "LastModified": "2026-04-14T17:23:54.1245112Z"
  }
]
D:\Programist\ksis\FileStorage>curl -X DELETE http://localhost:5166/images

D:\Programist\ksis\FileStorage>curl  http://localhost:5166/images/

D:\Programist\ksis\FileStorage>curl -v  http://localhost:5166/images/
*   Trying [::1]:5166...
* Connected to localhost (::1) port 5166
> GET /images/ HTTP/1.1
> Host: localhost:5166
> User-Agent: curl/8.4.0
> Accept: */*
>
< HTTP/1.1 404 Not Found
< Content-Length: 0
< Date: Tue, 14 Apr 2026 17:24:51 GMT
< Server: Kestrel
<
* Connection #0 to host localhost left intact

D:\Programist\ksis\FileStorage>
```

Рисунок 16 – Создание и удаление каталога

6 Копирования файлов в новое расположение

```
D:\Programist\ksis\FileStorage>echo Это содержимое текстового файла для лабораторной работы > demo.txt

D:\Programist\ksis\FileStorage>curl -X PUT --data-binary "@demo.txt" http://localhost:5166/original/source.txt
File source.txt uploaded successfully.
D:\Programist\ksis\FileStorage>curl -X PUT -H "X-Copy-From: /original/source.txt" http://localhost:5166/backup/copy.txt
File copied from 'original/source.txt' to 'backup/copy.txt'.
D:\Programist\ksis\FileStorage>curl http://localhost:5166/backup/
[
  {
    "Name": "copy.txt",
    "Type": "file",
    "Size": 58,
    "LastModified": "2026-04-14T17:28:06.5205113Z"
  }
]
```

Рисунок 17 – Копирование файла

http- трафик в Вайшарк представлен на рисунке 18.

7	131.652037	::1	::1	HTTP	282	PUT /docs/test2.txt HTTP/1.1 (application/x-www-form-urlencoded)
11	131.681701	::1	::1	HTTP	69	HTTP/1.1 200 OK
23	200.836704	::1	::1	HTTP	151	DELETE /picture HTTP/1.1
25	200.855017	::1	::1	HTTP	163	HTTP/1.1 404 Not Found
34	205.131123	::1	::1	HTTP	1182	GET / HTTP/1.1
42	205.147187	::1	::1	HTTP	351	GET /a3724ae021134b46b26ee0f1b48d4840/getScriptTag HTTP/1.1
46	205.147774	::1	::1	HTTP	69	HTTP/1.1 200 OK (text/plain)
50	205.152038	::1	::1	HTTP/J…	69	HTTP/1.1 200 OK , JSON (application/json)
52	207.305627	::1	::1	HTTP	1182	GET / HTTP/1.1
54	207.306603	::1	::1	HTTP	351	GET /a3724ae021134b46b26ee0f1b48d4840/getScriptTag HTTP/1.1
58	207.313095	::1	::1	HTTP	69	HTTP/1.1 200 OK (text/plain)
62	207.314760	::1	::1	HTTP/J…	69	HTTP/1.1 200 OK , JSON (application/json)
67	224.917227	::1	::1	HTTP	155	DELETE /picture.jpg HTTP/1.1
69	224.919357	::1	::1	HTTP	145	HTTP/1.1 204 No Content
75	227.162772	::1	::1	HTTP	1182	GET / HTTP/1.1
77	227.163639	::1	::1	HTTP	351	GET /a3724ae021134b46b26ee0f1b48d4840/getScriptTag HTTP/1.1
81	227.164111	::1	::1	HTTP	69	HTTP/1.1 200 OK (text/plain)
85	227.165000	::1	::1	HTTP/J…	69	HTTP/1.1 200 OK , JSON (application/json)
114	316.070130	::1	::1	HTTP	159	PUT /images/picture.jpg HTTP/1.1
118	316.076924	::1	::1	HTTP	69	HTTP/1.1 200 OK

Рисунок 18 – http-трафик

Содержимое http-запроса представлено на рисунке 19

```
▸ Frame 23: Packet, 151 bytes on wire (1208 bits), 151 bytes captured (1208 bits) on interface \Device\
▸ Null/Loopback
▸ Internet Protocol Version 6, Src: ::1, Dst: ::1
▸ Transmission Control Protocol, Src Port: 60367, Dst Port: 5166, Seq: 1, Ack: 1, Len: 87
▾ Hypertext Transfer Protocol
  ▸ DELETE /picture HTTP/1.1\r\n
    Host: localhost:5166\r\n
    User-Agent: curl/8.4.0\r\n
    Accept: */*\r\n
    \r\n
    [Response in frame: 25]
    [Full request URI: http://localhost:5166/picture]
```

Рисунок 19 – Содержимое http-запроса

Вывод: В ходе выполнения лабораторной работы было разработано удалённое файловое хранилище, предоставляющее REST API по протоколу HTTP. Приложение реализовано на языке C# с использованием веб-фреймворка ASP.NET Core Minimal API и веб-сервера Kestrel. Логически хранилище доступно по адресу http://localhost:5166, где путь в URL соответствует пути к файлу или каталогу внутри хранилища. Физически файлы сохраняются в подкаталоге Storage рабочей директории приложения.

В рамках работы реализованы все требуемые функции: загрузка файла с перезаписью методом PUT, получение содержимого файла и списка файлов каталога методом GET, получение метаданных файла через HTTP-заголовки методом HEAD, а также удаление файлов и каталогов методом DELETE. Дополнительно реализована функция копирования файлов с использованием заголовка X-Copy-From. Приложение корректно обрабатывает коды состояния HTTP. Реализована защита от Path Traversal атак путём нормализации пути и проверки выхода за пределы корневой директории хранилища.